Central de rotas

Checkpoint 5 | Grafos e caminho mínimo com Dijkstra

Implementação

A cidade foi representada por uma lista de adjacências usando dicionários. São sete vértices e doze ruas. Cada rua aparece nos dois sentidos, com o mesmo peso, porque o mapa é bidirecional.

O Dijkstra foi implementado em `grafo.py`, sem funções prontas de caminho mínimo. A origem começa com distância zero e os outros pontos, com infinito. Uma fila de prioridade com `heapq` retira o ponto de menor distância acumulada. Quando aparece um caminho mais barato até um vizinho, sua distância e seu predecessor são atualizados. No final, os predecessores permitem reconstruir o percurso do destino até a origem.

A interface está em `app.py`, e o desenho, em `mapa.py`. O Streamlit permite escolher os pontos e calcular a rota. O Matplotlib mostra os pesos e destaca o caminho com linhas mais grossas. As posições do desenho não estão em escala.

Testes do mapa original

Origem	Destino	Caminho encontrado	Soma	Total
A	G	A -> D -> G	2 + 3	5 km
A	C	A -> D -> B -> C	2 + 1 + 3	6 km
E	C	E -> D -> B -> C	4 + 1 + 3	8 km
G	A	G -> D -> A	3 + 2	5 km

A até G. Passar por D custa 5 km. A alternativa por D e F custa 6 km. A enumeração dos caminhos simples confirmou que nenhum percurso tem custo menor que 5 km.

A até C. Ir diretamente de A para B e depois C custa 7 km. Passar primeiro por D reduz o custo até B de 4 para 3 km, resultando em 6 km até C. Esse foi o menor valor na enumeração.

E até C. O percurso por D e B custa 8 km. Passar por F, D e B custa 9 km; seguir por D diretamente até C custa 10 km. A enumeração confirmou o mínimo de 8 km.

G até A. A troca entre origem e destino inverteu o percurso do primeiro teste e manteve os 5 km. Isso é coerente com as ruas bidirecionais e seus pesos iguais nos dois sentidos.

Acompanhamento do cálculo de A até G

Ponto retirado da fila	Distância	Atualizações de distância
A	0 km	B = 4, D = 2, E = 7
D	2 km	B = 3, C = 8, E = 6, F = 4, G = 5
B	3 km	C = 6
F	4 km	Nenhuma melhoria
G	5 km	Destino alcançado; cálculo encerrado

O predecessor de G é D, e o de D é A. A reconstrução produz A -> D -> G. Como os pesos são não negativos, a distância de um ponto retirado com o menor valor atual não pode ser melhorada por uma continuação mais longa. Por isso é possível encerrar quando o destino é retirado da fila, depois de descartar entradas desatualizadas.

Desafio adicional: mudança em uma rua

Foi incluída uma simulação que altera uma rua nos dois sentidos, preservando o mapa original para comparação.

Situação de D ↔ G	Menor rota de A até G	Total
Original, 3 km	A -> D -> G	5 km
Interditada	A -> D -> F -> G	6 km
Distância alterada para 10 km	A -> D -> F -> G	6 km

Com a interdição, a ligação direta entre D e G deixa de existir. Com o peso de 10 km, ela continua disponível, mas usá-la custaria 12 km desde A. Nas duas situações, passar por F é a melhor opção, com 6 km. A simulação mostra que uma alteração local pode mudar o caminho mínimo da entrega inteira.

Validação e limites

Os nove métodos de teste passaram. Além dos casos da tabela, os 49 pares de pontos foram comparados com uma busca que enumera todos os caminhos simples. Essa busca independente serve como conferência para este grafo pequeno.

Também foram testados pontos iguais, destino sem conexão, vértice inexistente, pesos inválidos e preservação do mapa original nas simulações. A interface avisa quando origem e destino são iguais. Se não houver rota, o algoritmo retorna infinito e uma lista vazia, e a interface apresenta uma mensagem de erro. Esse caso foi verificado com um grafo desconectado nos testes, pois nenhuma interdição isolada desconecta o mapa fornecido.

O uso da fila de prioridade tem custo O((V + E) log V) neste grafo simples e memória O(V + E), contando o grafo e a fila. O programa calcula distância em quilômetros; não considera tempo de viagem, trânsito ou sentido único.

As capturas da pasta `capturas` registram a interface, os quatro testes principais, as duas simulações e a validação de pontos iguais.